

UNIVERSITY OF LONDON

INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects



CM3070 PROJECT

FINAL PROJECT REPORT

AI-Powered Stock Recommendation System

Author: Jerrold Ng Yang Zhong

Student Number : 10262360

Date of Submission: 27th September 2026

Supervisor :

Repo: https://github.com/jerrold1907/FYP_Project.git

Abstract

Retail investors increasingly trade without professional advice, while most stock screeners ignore how much risk the individual investor can tolerate. This project builds and evaluates an AI financial advisor that combines a five-item risk questionnaire, a machine-learning signal that predicts whether a stock will rise by more than 10%, rise by up to 10% or fall over the next 30 trading days (Buy, Hold or Avoid), a rule-based suitability engine, and a chatbot whose language-model output is checked against the underlying data. The emphasis is on rigorous evaluation. A random train/test split inflated weighted F1 from 0.363 to 0.823 through temporal leakage. Under a purged chronological split the deployed logistic regression reaches 40.1% accuracy against 36.2% for always predicting the most common class: a statistically significant but small edge, which disappears when the analysis is repeated on 98 S&P 100 stocks. In walk-forward backtests the trades the model chose were profitable 62% of the time, well above the 47% expected from random entries, yet the strategy trailed buy-and-hold by 40 percentage points on average ($p = 0.043$), and accuracy fell below chance in the 2022 bear market. On nine real respondents the questionnaire's reliability (Cronbach's alpha 0.625) fell short of the 0.70 target, and the interface scored 73.6 on the System Usability Scale. The system works as a transparent, auditable decision-support prototype and a risk-awareness tool, but its predictions are not reliable enough to act on.

Contents

CHAPTER 1:	INTRODUCTION (417/1000 words)	5
1.1	Project Concept.....	5
1.2	Motivation.....	5
1.3	Aim and Objectives	6
1.4	Project Template	6
CHAPTER 2:	LITERATURE REVIEW (1924/2500 words)	7
2.1	Market Efficiency and the Limits of Prediction	7
2.2	Classical Machine Learning for Stock Prediction	7
2.3	Sequence Models: LSTM and Its Successors	8
2.4	Why Time Series Demands Different Treatment	9
2.5	Data Leakage and Evaluation Methodology	10
2.6	Statistical Comparison of Models and Strategies	10
2.7	Risk Profiling and Suitability	11
2.8	Language Models and Hallucination Control	12
2.9	Critical Synthesis	12
CHAPTER 3:	PROJECT DESIGN (1978/2000 words)	13
3.1	Domain, Users and Requirements	13
3.2	System Architecture.....	14
3.3	Technology Choices	15
3.4	Key Design Decisions.....	16
3.5	Data and Model Design	17
3.6	Risk Profiling and Suitability Design	18
3.7	Workplan	19
3.8	Evaluation Strategy.....	20
CHAPTER 4:	IMPLEMENTATION (1715/2500 words)	21
4.1	System Overview	21
4.2	Feature Engineering Pipeline.....	21
4.3	Model Training with Leakage Prevention	22
4.4	Risk Profiling and Suitability Engine	23

4.5 Walk-Forward Backtester	24
4.6 Conversational Interface	25
4.7 Web Application	26
4.8 Evaluation Code.....	28
CHAPTER 5: EVALUATION (2365/2500 words)	29
5.1 Data Leakage Discovery	29
5.2 Model Comparison	29
5.3 Stationarity Analysis.....	31
5.4 Backtesting Results.....	31
5.5 Market Regime Analysis	32
5.6 Scaling to 100 Stocks.....	33
5.7 Instrument and Filter Validation.....	34
5.8 Usability Evaluation	35
5.9 Testing	35
5.10 Critical Analysis	36
CHAPTER 6: CONCLUSION (834/1000 words)	37
CHAPTER 7: APPENDICES.....	40
CHAPTER 8: REFERENCES.....	49

CHAPTER 1: INTRODUCTION (417/1000 words)

1.1 Project Concept

This project develops an AI-powered financial advisor bot for the stock market, following Project Idea 4.2. The system combines machine learning-based stock signal prediction with a scenario-based investor risk questionnaire to produce personalised suitability recommendations.

The system comprises five components:

1. Risk Questionnaire — five scenario-based items, informed by Grable and Lytton (1999), classifying investors as Conservative, Moderately Conservative, Balanced, Growth or Aggressive.
2. Stock Recommender — a classifier trained on five years of daily data for twenty US equities across nine sectors, predicting Buy, Hold or Avoid from nine technical indicators, with the analysis repeated offline on 98 S&P 100 stocks to test whether the results hold at scale.
3. Suitability Engine — a deterministic 5×3 mapping from risk category and stock signal to a Suitable, Use Caution or Not Suitable rating, each with a written justification.
4. Walk-Forward Backtester — temporal validation measuring how the advisor's recommendations would have performed historically.
5. Conversational Interface — a chatbot answering questions about the recommendations, with a filter that rejects language-model output contradicting the underlying data.

1.2 Motivation

Most stock screeners issue generic signals without regard to an individual's risk tolerance, emotional comfort with volatility, or investment horizon, leaving a gap this project addresses. Retail investors increasingly trade without professional advice through platforms such as Robinhood, Trading 212 and eToro. Barber and Odean (2013)

document that individual investors systematically underperform, partly through behavioural bias and partly through mismatches between their risk tolerance and their holdings. Professional advice mitigates this but is economically inaccessible to small portfolios. This project examines whether an automated, transparent system can deliver a basic suitability assessment, and — importantly — whether such a system genuinely adds value when evaluated rigorously.

1.3 Aim and Objectives

Aim: To develop and rigorously evaluate an AI financial advisor bot that produces personalised stock suitability recommendations, integrating scenario-based risk profiling with machine learning prediction, and to establish through leakage-free evaluation and statistical inference whether its advice has measurable value.

#	Objective	Deliverable
Objective 1	Scenario-based risk questionnaire with five categories	src/risk_questionnaire.py
Objective 2	Technical indicator feature engineering	src/features.py
Objective 3	Train and compare classifiers under leakage-free evaluation	src/evaluation.py, retrain_model.py
Objective 4	Suitability engine mapping profiles to signals	src/suitability.py
Objective 5	Web application for non-technical users	Flask app, HTML/CSS/JS frontend
Objective 6	Walk-forward backtesting of recommendation quality	src/backtester.py
Objective 7	Statistical inference on all reported results	src/statistics_tests.py
Objective 8	Validate the questionnaire, matrix and LLM filter	src/instrument_validation.py, src/llm_evaluation.py

Table 1.1: Objectives and their deliverables

1.4 Project Template

Project Idea 4.2: Financial Advisor Bot

CHAPTER 2: LITERATURE REVIEW (1924/2500 words)

2.1 Market Efficiency and the Limits of Prediction

Any claim to predict returns must engage with the efficient market hypothesis. Fama (1970) argues that prices already reflect available information, so in the semi-strong form no strategy built from public data — which is precisely what technical indicators are — should earn abnormal risk-adjusted returns. Malkiel (2003) reviews the evidence three decades on and concedes that documented anomalies exist, but maintains that most vanish once transaction costs and the incentives of arbitrageurs are accounted for.

This framing matters for how the present project's results should be read. If markets are close to efficient, a weak out-of-sample edge is the expected outcome rather than a failure of implementation, and a large reported edge is grounds for suspecting a methodological error. That expectation shaped the decision to test for leakage (Section 5.1) rather than accept an unusually high accuracy at face value, and it explains why this review treats reported accuracies in the applied literature sceptically. It also justifies including transaction costs in the backtester, since Malkiel's central objection is that gross returns overstate what is achievable.

2.2 Classical Machine Learning for Stock Prediction

Random Forest, introduced by Breiman (2001), combines multiple decision trees trained on different samples and feature subsets. This reduces overfitting compared with a single tree and works well with technical indicators (Achelis, 2001) because it can capture complex relationships between features. More recent methods such as XGBoost (Chen and Guestrin, 2016) often perform even better on tabular data.

Patel et al. (2015) compared ANN, SVM, Random Forest and Naive Bayes on Indian stock indices, with Random Forest achieving the highest accuracy of 86.69%. However, the study does not report safeguards against the temporal leakage discussed in Section 2.5, so accuracies of this size may not reflect out-of-sample performance.

Basak et al. (2019) also found that ensemble models generally outperform individual classifiers, but noted that prediction is rarely linked to investor suitability. This is the gap addressed by the project's Suitability Engine. Nti et al. (2020) found that technical indicators are widely used and that shorter prediction horizons often perform better, supporting the use of a 30-day labelling window. Gu et al. (2020) showed that machine learning can improve asset prediction, but the gains are usually modest when tested properly out-of-sample. This provides a more realistic benchmark for the project.

2.3 Sequence Models: LSTM and Its Successors

A key limitation of tree-based models and logistic regression is that they treat each row as an independent observation. Indicators such as RSI and moving averages summarise past price movements, but they do not preserve the exact order of those movements. As a result, two stocks can have similar indicator values even if they reached them through very different price patterns.

Recurrent neural networks address this by passing information from one time step to the next through a hidden state. However, basic recurrent networks struggle with longer sequences because of vanishing or exploding gradients. Hochreiter and Schmidhuber (1997) introduced Long Short-Term Memory (LSTM) networks to reduce this problem. LSTMs use a cell state together with input, forget and output gates to control what information is stored, removed and passed forward. This allows information to be retained across longer sequences. Cho et al. (2014) later introduced the GRU, a simpler alternative with fewer parameters and similar performance in many tasks.

LSTMs have also been widely applied to financial prediction. Fischer and Krauss (2018) used LSTMs on S&P 500 stocks from 1992 to 2015 and found that they achieved higher directional accuracy than Random Forest, deep neural networks and logistic regression. However, the advantage was stronger during the earlier and more volatile part of the dataset. Sezer et al. (2020) similarly found that LSTM-based models are among the most commonly used deep learning approaches in financial time-series research, while noting that differences in evaluation methods make direct comparisons between studies difficult.

Later research has been more mixed. Siarni-Namini et al. (2019) showed that the performance of LSTM and bidirectional LSTM depends strongly on the characteristics of the dataset. Transformers, introduced by Vaswani et al. (2017), use attention instead of recurrent connections, while Lim et al. (2021) developed the Temporal Fusion Transformer specifically for multi-horizon time-series forecasting. However, Zeng et al. (2023) showed that simple linear models can match or outperform more complex architectures on several time-series benchmarks, suggesting that more complex models do not always lead to better predictions.

The evidence is therefore mixed: sequence models can exploit temporal order, but simple linear models remain competitive on many benchmarks. Where predictive performance is similar, Rudin (2019) argues that high-stakes decisions should use inherently interpretable models rather than post-hoc explanations of black boxes, because such explanations can misrepresent what the model actually does. This motivates comparing an LSTM with logistic regression under identical conditions rather than assuming either is superior (Section 3.5).

2.4 Why Time Series Demands Different Treatment

Random Forest and logistic regression generally assume that observations are independent and come from a stable distribution, an assumption that time-series data such as financial prices routinely violates.

Financial observations are not fully independent because returns can show autocorrelation and periods of high or low volatility. The distribution also changes over time. A stationary series has a stable mean, variance and autocovariance, while stock prices are usually non-stationary because they trend. Dickey and Fuller (1979) developed the ADF test, where non-stationarity is the null hypothesis, while Kwiatkowski et al. (1992) proposed the KPSS test, which uses stationarity as the null. Using both therefore provides stronger evidence, as done in Section 5.3.

Non-stationarity is especially problematic for tree models. Decision trees learn fixed thresholds from the training data and cannot extrapolate well beyond the values they have

seen. If a price rises outside the training range, future observations may repeatedly fall into the same leaf, reducing the model's ability to distinguish them. Linear models can extrapolate, although this may still be unreliable. A common solution is to transform features using differencing or ratios to make them more stationary. Section 5.3 tests which features in this project require these transformations.

2.5 Data Leakage and Evaluation Methodology

López de Prado (2018) argues that financial machine learning is especially vulnerable to data leakage. He recommends purging observations whose label periods overlap with the test set and adding an embargo period to reduce the effect of serial correlation. Since this project uses 30-day forward returns, nearby observations share part of the same future period. A random split can therefore allow training data to contain information related to test outcomes. For this reason, the backtester in Section 5.4 uses a chronological walk-forward approach instead.

Kaufman et al. (2012) define leakage as using information that would not have been available at prediction time, and Kapoor and Narayanan (2023) found leakage to be common across 294 studies. The choice of assets can bias results in a similar way: Brown et al. (1992) show that studying only the assets that survive to the end of a sample overstates their measured performance, because failed and delisted firms drop out of it.

2.6 Statistical Comparison of Models and Strategies

Comparing model performance using accuracy alone does not show whether the difference is meaningful. Dietterich (1998) recommends paired statistical tests when models are evaluated on the same observations, and McNemar's test (McNemar, 1947) is suitable for comparing classification errors between two models. For comparisons across several market regimes, ANOVA and the non-parametric Kruskal-Wallis test (Kruskal and Wallis, 1952) are used, with post-hoc testing where needed. Demšar (2006) cautions that comparisons of classifiers across studies are often statistically unsound, which motivated the paired and rank-based tests used throughout this evaluation rather than a

single point-estimate comparison; the ANOVA itself follows Fisher's (1925) original formulation.

Strategy performance also requires measures of uncertainty. Lo (2002) showed that Sharpe ratio estimates can be unreliable for short or serially correlated return series. Confidence intervals are therefore reported for measures such as Sharpe ratio, accuracy and win rate, with Wilson (1927) intervals used for proportions because they perform better with small samples. These methods are applied in Sections 5.2–5.5 to determine whether observed differences are statistically meaningful rather than simply due to random variation. Harvey and Liu (2015) warn that backtested performance is routinely overstated by multiple testing across parameter choices and asset universes, which is why this project fixes its label thresholds and backtest parameters in advance and reports every candidate model rather than only the best (Section 3.5).

2.7 Risk Profiling and Suitability

Grable and Lytton (1999) developed a widely used 13-item risk-tolerance questionnaire covering factors such as reaction to losses, investment horizon and financial knowledge. Questionnaire reliability can be measured using Cronbach's alpha (Cronbach, 1951), with 0.70 commonly treated as an acceptable level of internal consistency (Nunnally and Bernstein, 1994). Because alpha rises with the number of items, a shorter scale must rely on each item measuring the construct well. Section 5.7 therefore measures the reliability of the project's risk questionnaire instead of assuming that its questions form a reliable scale. Kannadhasan (2015) also found that factors such as age and investment experience influence risk tolerance.

More recent research has examined automated financial advice. D'Acunto and Rossi (2021) note that robo-advisers can improve diversification, but their suitability processes are rarely tested in detail. This project addresses this by using a clear, rule-based Suitability Engine rather than relying on an opaque model to decide whether an investment matches an investor, echoing the suitability principle in industry regulation: FINRA's (2014) Rule 2111 requires that a recommendation be suitable for a customer's

investment profile, a standard a deterministic, auditable mapping is better placed to satisfy than a learned one.

2.8 Language Models and Hallucination Control

Ji et al. (2023) distinguish between intrinsic hallucinations, where generated text contradicts the source information, and extrinsic hallucinations, where unsupported information is added. Intrinsic hallucinations are particularly important in financial advice because incorrectly reporting a stock's suitability or prediction could mislead the user.

Huang et al. (2025) review several approaches to reducing hallucinations, including training methods, generation controls and post-generation verification. Retrieval-augmented generation (Lewis et al., 2020) can also ground responses in external information, but it does not completely remove the risk of incorrect outputs.

These findings suggest a design in which factual content comes from structured model output and the language model only rephrases it, with a check after generation that rejects any contradiction (Section 3.4). Because such a filter can itself make mistakes, its precision and recall need to be measured rather than assumed (Section 5.7).

2.9 Critical Synthesis

Several gaps in the literature directly motivate this project. First, many prediction studies focus on accuracy without linking predictions to investor suitability. Second, data leakage remains common and can greatly inflate reported results (Kapoor and Narayanan, 2023).

Third, prediction accuracy does not by itself make a trading strategy profitable: transaction costs can erase a statistical edge (Malkiel, 2003) and backtests tend to overstate performance (Harvey and Liu, 2015), so signal quality and investment value need to be evaluated separately. Fourth, model selection is often based on previous studies rather than testing whether the model suits the statistical properties of the data, which motivates the stationarity analysis in Section 5.3. Finally, risk profiling and other

user-facing parts of robo-advisory systems are often not properly validated, motivating the reliability and hallucination-filter tests in Section 5.7.

Overall, the contribution of this project is not a new machine-learning architecture. Instead, it combines investor profiling, stock prediction and suitability assessment with stricter evaluation methods designed to produce more reliable results, even when those results show limited predictive performance.

CHAPTER 3: PROJECT DESIGN (1978/2000 words)

This chapter describes the design as built. Two parts changed in response to results: the random train/test split was replaced by a purged chronological split in week 9, after the leakage in Section 5.1 was found, and stationarity testing was added in week 11.

3.1 Domain, Users and Requirements

Domain: personal finance and retail investment decision support.

Target users are self-directed retail investors on consumer trading platforms, with basic equity knowledge but little technical-analysis expertise, who want guidance matched to their risk tolerance and explained rather than merely asserted.

User stories:

- US1: As a cautious investor I want to answer a few questions about my comfort with risk and see which stocks suit me, so I avoid exposure I would not tolerate.
- US2: As a growth-oriented investor I want to see a stock's chart, key indicators and the reasoning behind its rating, so I can judge the advice rather than merely accept it.
- US3: As a sceptical user I want evidence of how the recommendations performed historically, including where they failed, so I can calibrate my trust.

Table 3.1 turns the stories into requirements, each traced to its source and to where it is evaluated. Users were not consulted during design; they took part in the usability evaluation (Section 5.8), where their feedback and the resulting changes are reported.

ID	Requirement	Source	Evaluated in
FR1	Classify risk tolerance into five categories from a short questionnaire	US1; Grable and Lytton (1999)	5.7
FR2	Predict Buy, Hold or Avoid for each stock from technical indicators	US1; Patel et al. (2015)	5.1–5.6
FR3	Rate each (risk category, signal) pair, with a written reason	US1, US2; FINRA (2014)	5.7
FR4	Show each stock's chart, indicators and rating rationale	US2	4.7, 5.8
FR5	Report historical performance, including failures, with uncertainty	US3; Harvey and Liu (2015)	5.4–5.6
FR6	Answer questions without stating anything the data contradicts	US2; Ji et al. (2023)	5.7
NFR1	Every rating traceable to an explicit rule	FINRA (2014); Rudin (2019)	5.7
NFR2	No temporal leakage in any evaluation	López de Prado (2018)	5.1
NFR3	Questions processed locally, not by third-party services	US1	3.3
NFR4	SUS score above 68	Sauro and Lewis (2016)	5.8

Table 3.1: Requirements, their sources and where they are evaluated

3.2 System Architecture

Figure 3.1 shows the components and data flows. A browser front end calls a Flask server through a JSON API. The recommendation path runs from Yahoo Finance prices through feature engineering and the Logistic Regression model to the suitability engine, which combines each signal with the user's risk category. The backtester and chatbot sit beside this path, and offline scripts train the model on historical prices.

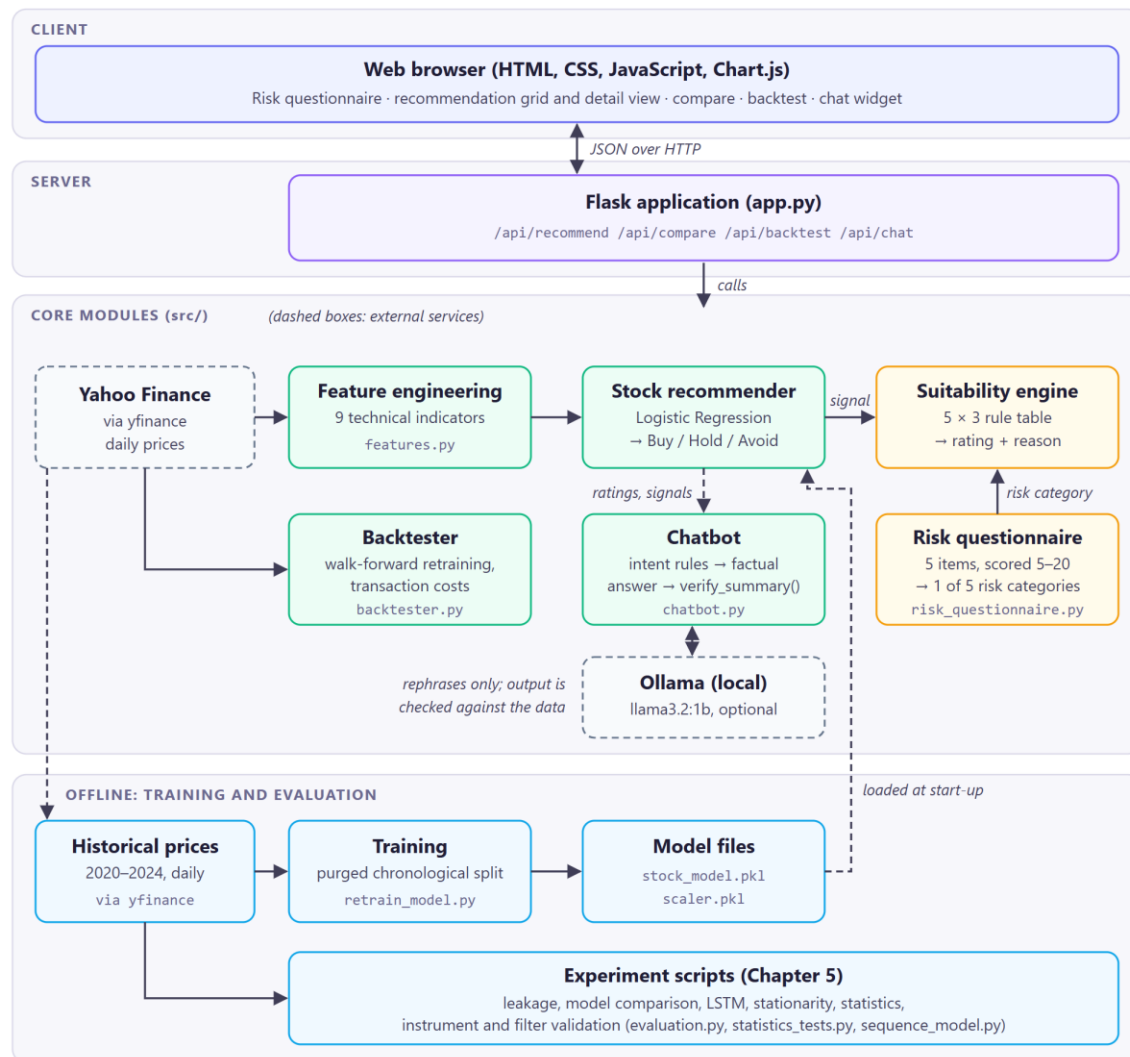


Figure 3.1: Component architecture. Dashed boxes are external services; dashed arrows are indirect or start-up flows.

3.3 Technology Choices

Table 3.2 lists each technology with the alternatives considered.

Technology	Role	Alternatives considered	Reason for the choice
Python 3.13	Implementation language	R; Java	One language for modelling, statistics and the web server
scikit-learn	Classical models, baselines	XGBoost; LightGBM	One interface for all classical models and baselines; boosting was unnecessary for testing whether the features carry signal
PyTorch	LSTM baseline	TensorFlow/Keras	Explicit training loop made time-ordered early stopping and seeding easy to control

Technology	Role	Alternatives considered	Reason for the choice
statsmodels, SciPy	Statistical tests	Hand-written tests	Reference implementations, checked against published values (Section 5.9)
Flask	Web server, JSON API	Django; FastAPI	Three pages, four JSON endpoints and no database: Django's ORM and admin would go unused, and FastAPI's asynchronous handling gains little when requests wait on synchronous downloads
Chart.js	Charts	Plotly; D3.js	Small library that draws in the browser from JSON
Ollama, llama3.2:1b	Rephrasing chatbot answers	Larger local models; cloud LLM APIs	Runs on a laptop CPU and keeps questions local. Its errors are contained by design, not size: it never originates facts, a filter rejects contradictions and the factual answer is always shown. A larger model would err less, not never, at several times the cost
yfinance	Daily prices	Alpha Vantage; paid feeds	Free, keyless, full history (Aroussi, 2024) and adequate for an academic prototype, but not for a live service: it is unofficial, has no service guarantee, and adjusted prices are revised retroactively when dividends are paid
Hypothesis	Property-based tests	Example-based tests only	Checks invariants, such as the matrix's ordering, across generated inputs

Table 3.2: Technology choices and the alternatives considered

3.4 Key Design Decisions

Table 3.3 traces each main design decision to the literature in Chapter 2 and to the section that tests it.

Decision	Alternative rejected	Rationale	Literature (Chapter 2)	Tested in
1. Purged chronological split	Random stratified split	Rolling features and overlapping 30-day labels leak the future into a random split	López de Prado (2018); Kaufman et al. (2012); §2.5	5.1
2. Every metric against naive baselines	Raw metrics alone	On an imbalanced target a score means little without a trivial predictor's floor; efficient markets allow only a small edge	Fama (1970); Malkiel (2003); §2.1	5.2
3. Intervals and paired tests	Point estimates	Few trades per stock make point estimates unreliable; shared test rows need paired tests	Dietterich (1998); Wilson (1927); Lo (2002); §2.6	5.2–5.6
4. Rule-based suitability matrix	Learned mapping from profile to rating	Fifteen explicit rules can be audited and tested exhaustively, as suitability requires	FINRA (2014); D'Acunto and Rossi (2021); Rudin (2019); §2.3, §2.7	5.7
5. Scenario-based questionnaire	Single risk slider	Scenarios cover several facets of risk tolerance and make reliability measurable	Grable and Lytton (1999); Cronbach (1951); §2.7	5.7
6. Language model only rephrases; output checked	Model writes answers; prompt instructions alone	Small models mis-attribute facts, so correctness must not depend on the model	Ji et al. (2023); Huang et al. (2025); §2.8	5.7

Decision	Alternative rejected	Rationale	Literature (Chapter 2)	Tested in
7. Transaction costs in every backtest	Gross returns	Gross returns overstate what is achievable	Malkiel (2003); Harvey and Liu (2015); §2.1, §2.6	5.4
8. LSTM compared with row-wise models	Assuming one model class	Evidence on sequence models is mixed	Fischer and Krauss (2018); Zeng et al. (2023); §2.3	5.2
9. Stationarity tested, not assumed	Using features unchecked	Non-stationary inputs break both model families' stable-distribution assumption	Dickey and Fuller (1979); Kwiatkowski et al. (1992); §2.4	5.3

Table 3.3: Design decisions, the literature behind them and where they are tested

3.5 Data and Model Design

Stock universe. The main study uses 20 large US stocks chosen by hand, widened from an initial ten, mostly technology, stocks so that defensive (healthcare, utilities, consumer staples), cyclical (energy, industrials) and highly volatile (semiconductor) sectors are represented, which the regime analysis in Section 5.5 relies on. This has two weaknesses. Hand-picking companies that are large today introduces survivorship bias (Brown et al., 1992): they are known to have survived, and some to have boomed, which inflates buy-and-hold returns. And twenty stocks is small for a real recommender. In response to feedback, the core analyses were later repeated on the S&P 100 (Section 5.6), a universe defined by rule (Wikipedia, 2026). It is still survivorship-biased, since it uses today's members; free data sources hold no histories for delisted stocks, so a point-in-time list was not feasible.

Features. The nine inputs are the indicators most used in the studies reviewed (Achelis, 2001; Patel et al., 2015; Nti et al., 2020): closing price, daily return, 5-, 20- and 50-day moving averages, 20-day volatility, volume, 14-day RSI and MACD. The aim was to test whether this standard set carries signal. It sits uneasily with Section 2.4, since price levels and moving averages trend and are non-stationary. This tension was recognised only after the first models were built, so stationarity was measured (Section 5.3) rather than assumed, and replacing price levels with ratios such as $\text{price}/\text{ma}_{50}$ is the first extension (Section 5.10).

Labels. A row is labelled Buy if the return over the next 30 trading days exceeds +10%, Avoid if it is negative and Hold otherwise. The 30-day horizon follows Nti et al. (2020), who found shorter horizons more predictable, and suits a monthly review. The 0% boundary separates a gain from a loss. The +10% threshold was a judgement made before any model was trained: a gain large enough to matter after costs that still leaves Buy about a fifth of the rows (22%). It was not tuned; a later check (Appendix H) repeats the evaluation with thresholds of +5%, +7.5% and +15%.

Model selection criteria. Candidate models are compared on macro F1, which weights the rare Buy class as heavily as the others, alongside accuracy, balanced accuracy and McNemar tests. Choosing which model to deploy uses three further criteria that a leaderboard does not capture:

1. Significance: it must beat both naive baselines on the same test rows (McNemar, $p < 0.05$).
2. Transparency: an auditor must be able to see how the inputs led to a signal, because a suitability rating rests on it (FINRA, 2014; Rudin, 2019).
3. Stability: retraining on the same data must give the same model, because the backtester retrains it for every window (about 220 times per run) and a result that changes with the random seed cannot be audited.

Among models meeting the first criterion, the most transparent and stable is deployed unless another has a significantly lower error rate (applied in Section 4.3).

3.6 Risk Profiling and Suitability Design

Questionnaire. Five scenario items, each scored 1–4, cover facets of risk tolerance identified by Grable and Lytton (1999): reaction to a loss, investment goal, time horizon, emotional comfort with volatility and the risk–return trade-off. Their instrument has 13 items; five were used because the questionnaire is the application's first screen, where each extra question costs completions, and each item covers a different facet. The price is reliability, which falls as items are removed (Section 2.7). Totals of 5–20 are split into five bands (5–8, 9–11, 12–14, 15–17, 18–20): sixteen totals cannot divide evenly by five, so the lowest band is four points wide, placing borderline low scorers in the Conservative

band, the cautious direction for a suitability rule. The bands are uncalibrated; Section 5.7 measures reliability and band sensitivity.

Suitability matrix. Table 3.4 gives the full mapping. Two ordering rules shaped it: permissiveness never falls as risk tolerance rises or as the signal strengthens, and no profile is told that an Avoid signal is Suitable. `src/suitability.py` holds the explanation shown to the user for each of the 15 cells.

Risk category	Buy	Hold	Avoid	Reasoning
Conservative	Use Caution	Not Suitable	Not Suitable	Capital preservation first; even a positive signal carries unaffordable downside
Moderately Conservative	Use Caution	Not Suitable	Not Suitable	Prefers stability; a neutral outlook does not justify the risk
Balanced	Suitable	Use Caution	Not Suitable	Accepts moderate risk for a positive outlook, with care on a neutral one
Growth	Suitable	Suitable	Use Caution	Can hold through uncertainty; a negative signal calls for care, not exclusion
Aggressive	Suitable	Suitable	Use Caution	Highest tolerance, but still cautioned on a negative signal

Table 3.4: The suitability matrix

3.7 Workplan

Week	Tasks	Milestone
1-2	Project setup, validation, risk questionnaire	Core modules
3-4	Suitability engine, feature engineering, property tests	Deterministic components tested
5-6	Data acquisition, model training	Models trained
7-8	Backtester, Flask application, chatbot	Prototype working
9	Leakage diagnosis and correction; retraining	Evaluation methodology corrected
10	Statistical inference layer; regime analysis	Results carry intervals and tests
11	LSTM baseline, stationarity diagnostics	Model choice evidenced
12	Instrument and LLM filter validation	Human-facing components validated
13-14	User evaluation with real participants; SUS	Usability data collected
15-16	Final report, video, submission	Submission ready

Table 3.5: Workplan by week

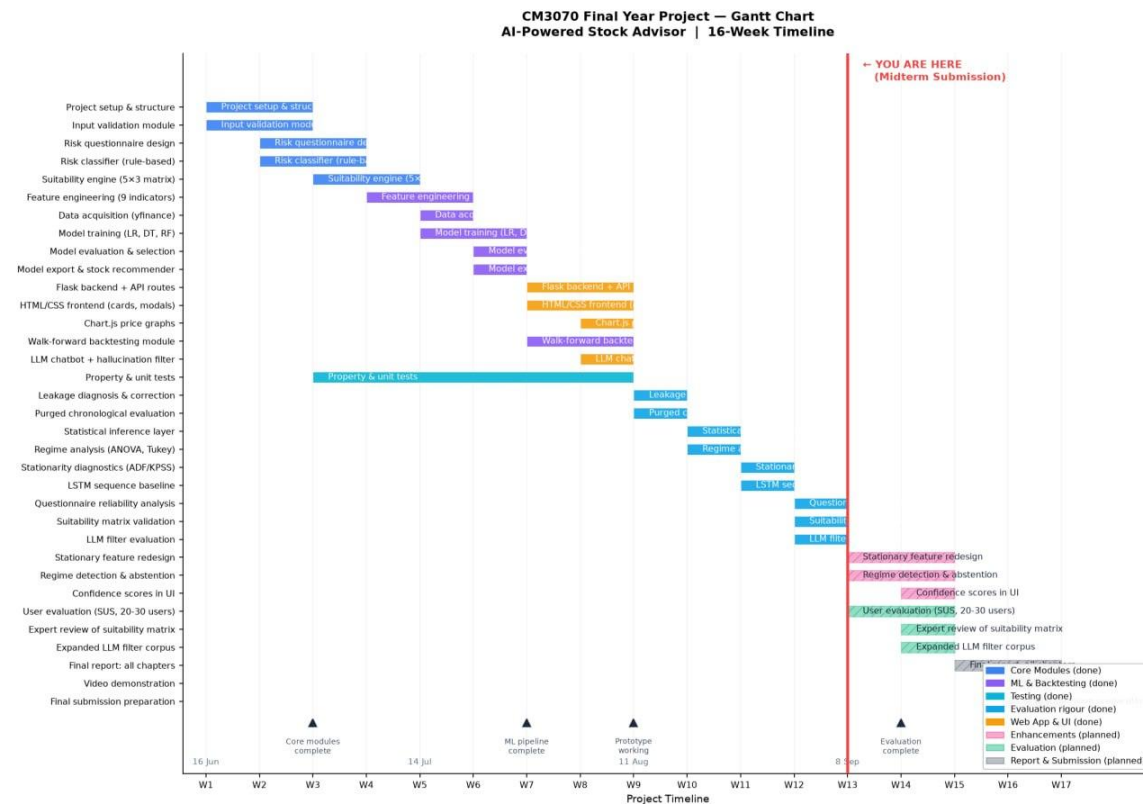


Figure 3.2: Gantt chart of the workplan

3.8 Evaluation Strategy

Classifiers are compared on macro F1 with accuracy, weighted F1 and balanced accuracy alongside, all against naive baselines (Table 3.3); trading uses win rate, return, Sharpe ratio and drawdown against buy-and-hold; and the user-facing components are evaluated separately with the techniques in Table 3.6.

Technique	Applied to	Criterion
Wilson intervals	Accuracy, win rate	95% intervals, reliable at small n (Wilson, 1927)
Sharpe ratio interval	Risk-adjusted return	Interval excludes zero before claiming skill (Lo, 2002)
ANOVA, Kruskal-Wallis	Market regimes	$p < 0.05$; rank test preferred because per-row accuracy is binary
Cronbach's alpha	Questionnaire	$\alpha \geq 0.70$ (Nunnally and Bernstein, 1994)
Monotonicity checks	Suitability matrix	Suitability never falls as risk tolerance or signal strength rises
Filter recall	Contradiction filter	High, so few incorrect statements reach users
System Usability Scale	Interface	Score above 68 (Sauro and Lewis, 2016)

Table 3.6: Further evaluation techniques and their criteria

CHAPTER 4: IMPLEMENTATION (1715/2500 words)

4.1 System Overview

The delivered system is a Flask web application that takes a user from risk profiling to personalised stock suitability recommendations in a single session. It covers 20 US equities across nine GICS sectors. The prediction models were developed using 23,560 daily observations from January 2020 to December 2024. The codebase is structured as a Python package with separate modules for features, prediction, suitability, backtesting, chatbot and the web layer, supported by 353 automated tests; Figure 3.1 shows how they fit together. Only the Logistic Regression model and the 20-stock universe are deployed; the model comparison, the S&P 100 check and the statistical analyses in Chapter 5 run offline (Section 4.8).

4.2 Feature Engineering Pipeline

The `compute_features()` function in `src/features.py` transforms raw OHLCV data into nine technical indicators used as model inputs:

1. `close_price` — the adjusted closing price
2. `daily_return` — percentage change from the previous close
3. `ma_5` — 5-day simple moving average
4. `ma_20` — 20-day simple moving average
5. `ma_50` — 50-day simple moving average
6. `volatility` — 20-day rolling standard deviation of daily returns
7. `volume` — daily trading volume
8. `RSI` — 14-day relative strength index
9. `MACD` — difference between 12-day and 26-day exponential moving averages

Labelling uses 30-day forward returns computed from the close price: Buy above +10%, Hold from 0% to +10% and Avoid below 0%, giving roughly 22% Buy, 39% Hold and 39% Avoid (Section 3.5 gives the reasoning).

4.3 Model Training with Leakage Prevention

The training pipeline implements three safeguards against data leakage:

1. Purged chronological split: the dataset is ordered by date and split at the 80th percentile. No shuffling is performed. The test set begins at 2023-12-08, giving train $n = 18,240$ and test $n = 4,720$.
2. 30-day purge window: the final 30 days of the training set (600 rows) are discarded, because their 30-day forward return labels overlap the beginning of the test period. This prevents the most damaging form of leakage identified by López de Prado (2018).
3. StandardScaler fitted on training data only: feature scaling parameters are computed exclusively from the training partition and then applied to the test set, preventing information from the test distribution from entering the model through normalisation statistics.

The purge itself is a single date comparison in `src/evaluation.py` (`purged_chronological_split()`), reproduced below because Section 5.1's leakage correction depends entirely on it:

```
split_pos = int(round(len(unique_dates) * (1.0 - test_size)))
split_date = pd.Timestamp(unique_dates[split_pos])

# Purge: training must end label_horizon trading days before the split.
purge_pos = max(0, split_pos - label_horizon)
purge_start = pd.Timestamp(unique_dates[purge_pos])

test_mask = date_series >= split_date
train_mask = date_series < purge_start
```

Splitting on unique trading dates rather than row count keeps every ticker's rows for a given day on the same side of the split, so no ticker can straddle the boundary, and subtracting `label_horizon` (30) trading days from the cut-off before selecting training rows is what stops a training label from seeing into the test period. This eight-line mechanism is what turned a 0.823 weighted F1 into 0.363.

Logistic Regression was chosen for deployment by applying the criteria in Section 3.5 to the results in Section 5.2:

- **Significance.** It beats both naive baselines (McNemar $p < 0.001$) and has the highest accuracy, 0.401. Its error rate is significantly lower than that of the reported LSTM run ($p = 0.017$) and not significantly different from the Random Forest's or the decision tree's ($p = 0.06$).
- **Transparency.** It is linear: each class score is a weighted sum of the nine standardised indicators, and the weights can be read directly. The largest weight for Buy is on recent volatility (+0.33), so the model favours Buy after volatile periods. The four price-level features are almost collinear, which blurs their individual weights, another reason for Extension 1. Neither the Random Forest nor the LSTM offers so direct an account.
- **Stability.** Fitting it is a convex problem, so retraining on the same data always gives the same model. The LSTM does not: across five random seeds its accuracy ranged from 0.380 to 0.408, either side of Logistic Regression's.

The cost is explicit. The LSTM's macro F1 is higher in every seed (0.365–0.381 against 0.337) because it finds more of the Buy class (recall 0.26–0.37 against 0.07), helped by its class-weighted loss (Section 4.8). Deploying Logistic Regression trades Buy recall for transparency and reproducibility, a trade the suitability setting favours (Rudin, 2019). Section 5.10 records the low Buy recall as the deployed system's main weakness, and Extension 2 would give Logistic Regression the same weighting.

4.4 Risk Profiling and Suitability Engine

The risk questionnaire presents five scenario-based questions, each scored 1 to 4 from most conservative to most aggressive. The scenarios cover loss reaction, investment goal, time horizon, emotional comfort with volatility and the risk–return trade-off. Total scores range from 5 to 20 and map deterministically to five risk categories:

Score Range	5-8	9-11	12-14	15-17	18-20
Risk Category	Conservative	Moderately Conservative	Balanced	Growth	Aggressive

Table 4.1: Questionnaire score bands and risk categories

The suitability engine implements the matrix in Table 3.4 as a lookup table keyed by (risk category, signal) that returns the rating and its written explanation (Figure 4.1). Its two ordering properties are checked by property-based tests and in Section 5.7.

```
MAPPING: dict[tuple[str, str], str] = {
    # Conservative: very risk-averse, only cautious on strong Buy
    ("Conservative", "Buy"): "Use Caution",
    ("Conservative", "Hold"): "Not Suitable",
    ("Conservative", "Avoid"): "Not Suitable",
    # Moderately Conservative: slightly more open but still cautious
    ("Moderately Conservative", "Buy"): "Use Caution",
    ("Moderately Conservative", "Hold"): "Not Suitable",
    ("Moderately Conservative", "Avoid"): "Not Suitable",
    # Balanced (also accepts legacy "Moderate"): accepts moderate risk
    ("Balanced", "Buy"): "Suitable",
    ("Balanced", "Hold"): "Use Caution",
    ("Balanced", "Avoid"): "Not Suitable",
    ("Moderate", "Buy"): "Suitable",
    ("Moderate", "Hold"): "Use Caution",
    ("Moderate", "Avoid"): "Not Suitable",
    # Growth: accepts significant risk for returns
    ("Growth", "Buy"): "Suitable",
    ("Growth", "Hold"): "Suitable",
    ("Growth", "Avoid"): "Use Caution",
    # Aggressive: highest risk tolerance
    ("Aggressive", "Buy"): "Suitable",
    ("Aggressive", "Hold"): "Suitable",
    ("Aggressive", "Avoid"): "Use Caution",
}
```

Figure 4.1: The suitability mapping in src/suitability.py

4.5 Walk-Forward Backtester

The backtester in src/backtester.py implements a rolling-window protocol that simulates realistic deployment conditions:

- Training window: 500 trading days
- Test window: 60 trading days
- Step: the window advances by 60 days after each evaluation
- Per-window retraining: a fresh model is fitted on each training window, preventing look-ahead bias
- Positions: a Buy signal takes a full position held until the signal changes; a Hold signal takes a half position that stays open while Hold signals continue; Avoid holds cash
- Transaction costs: 0.1% on entry and 0.1% on exit, charged only when a position is opened or closed, following Malkiel's (2003) argument that gross returns overstate achievable performance

For each ticker the backtester records every trade's outcome, the strategy and buy-and-hold returns, the excess return, the Sharpe ratio and the maximum drawdown. The /backtest page runs it on the tickers a user chooses (Section 4.7);

exp03_statistical_analysis.py runs it on all 20 and adds the intervals and tests reported in Section 5.4, pooling trades across tickers for the win-rate claim.

4.6 Conversational Interface

The chatbot layer provides a natural-language interface to the structured recommendations. It operates in three stages:

1. Intent classification: a rule-based classifier recognises eight intents (compare, explain_recommendation, explain_risk, risk_analysis, stock_info, best_pick, portfolio_overview and general_help) from keyword patterns and the tickers or company names mentioned.
2. Deterministic handlers: each intent maps to a handler that constructs a factual answer directly from model predictions and suitability ratings, ensuring no information is invented.
3. Optional LLM rephrasing: if Ollama is available, the factual answer is passed to llama3.2:1b for natural-language rephrasing. The verify_summary() function then checks the rephrased output against the structured data. If any sentence attributes a specific rating or signal to the wrong ticker, the rephrased version is discarded and the deterministic answer is returned instead. This contradiction filter addresses an observed failure where the model attributed META's suitability rating to AAPL.

This behaviour was found empirically, not designed up front. Asked to compare AAPL and META, llama3.2:1b opened its summary with “AAPL is not suitable for your risk profile” when the Suitability Engine had rated META Not Suitable and AAPL Suitable: it rephrased each rating correctly but swapped the tickers when summarising both. An explicit prompt instruction not to swap tickers reduced but did not eliminate the error across repeated runs, so verify_summary() (src/chatbot.py) was written as a deterministic check that treats the model's output as untrusted:

```
for sentence in re.split(r"(?<=[.!?])\s+", summary):
    named = [t for t in facts if re.search(rf"\b{t}\b", sentence, re.I)]
    # Ambiguous sentences (0 or 2+ tickers) cannot be attributed safely.
    if len(named) != 1:
        continue
```

```

fact = facts[named[0]]
lowered = sentence.lower()

# "not suitable" must be tested before "suitable" (substring overlap).
for term in ("not suitable", "use caution", "suitable"):
    if term in lowered:
        if fact["rating"] and term != fact["rating"]:
            return False
        break

```

The function is deliberately conservative: a sentence naming more than one ticker is skipped, because attribution cannot be resolved safely. This trades recall for precision, discarding some sound summaries rather than letting a contradiction through. Figure 4.2 shows the resulting pipeline; Section 5.7 measures the filter's error rates.

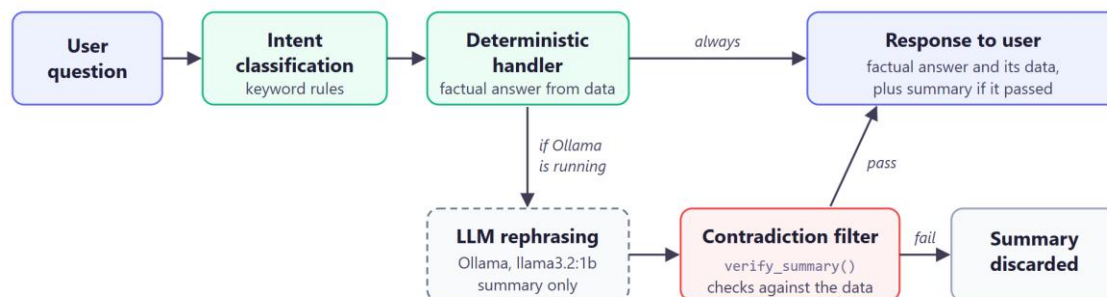


Figure 4.2: Chatbot pipeline. The factual answer is always shown; the language-model summary is added only if it passes the contradiction filter.

4.7 Web Application

The Flask application serves three pages, backed by four JSON endpoints

(/api/recommend, /api/compare, /api/backtest and /api/chat):

- / (index): risk questionnaire followed by a responsive grid of stock recommendations. Each card expands into a modal with chart, indicators, strengths, risks and justification. (Figures 4.3–4.4; Appendix A)
- /compare: side-by-side comparison of two or three stocks with their signals, indicators and price charts.
- /backtest: runs the walk-forward backtest for chosen tickers, model and windows, charting the strategy against buy-and-hold alongside aggregate metrics, including the combined strategy's Sharpe ratio with a 95% interval, and per-ticker results (Figure B1).

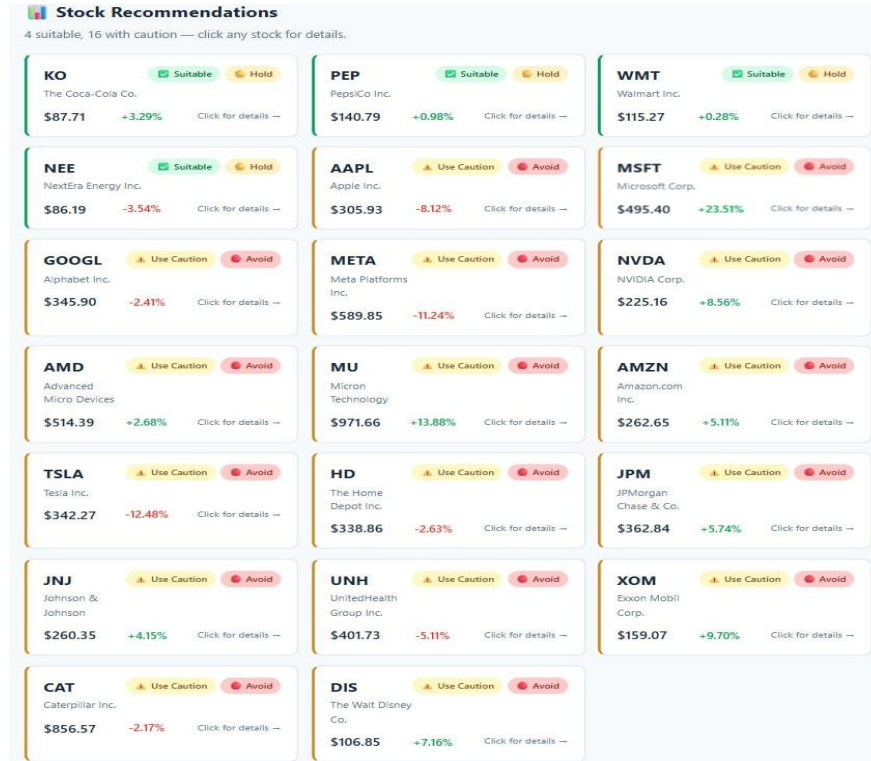


Figure 4.3: Main recommendation page after completing the risk questionnaire. Stocks are sorted by suitability for user's risk profile

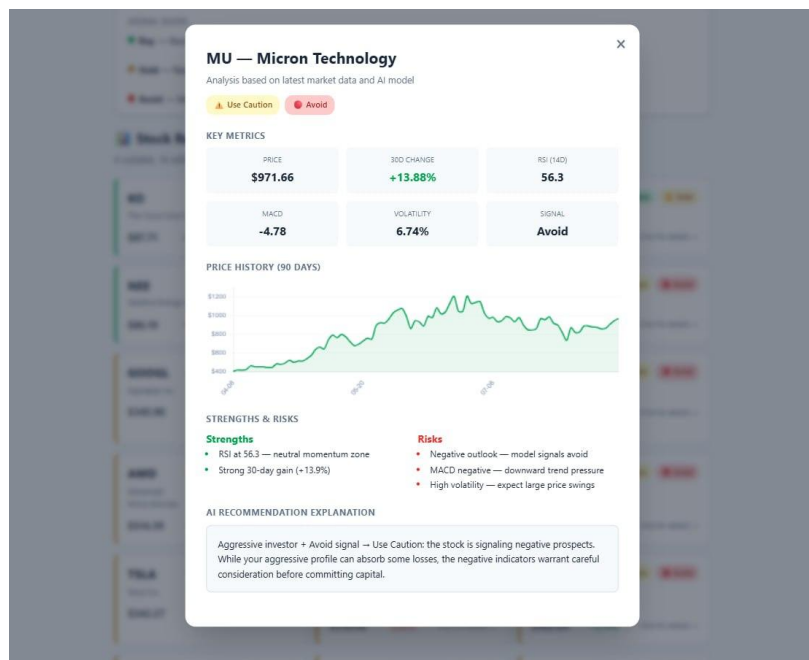


Figure 4.4: Stock detail modal showing price chart, technical indicators and the suitability explanation for an Aggressive user

The frontend uses Chart.js for interactive price charts rendered client-side, a floating chat widget available on all pages for conversational queries, and a responsive CSS grid that adapts from desktop to mobile.

4.8 Evaluation Code

The analyses in Chapter 5 come from seven experiment scripts (Appendix F). They share the evaluation modules below and read prices from a dated snapshot (`src/market_data.py`), so every run sees the same data:

- `src/statistics_tests.py`: Wilson intervals, McNemar's test, ANOVA with Tukey post-hoc tests, Kruskal-Wallis, sample-size calculations and Lo's (2002) Sharpe-ratio interval. Its chance-matched test gives each trade its own chance of winning, the share of entry days in the same test window that would have profited over the same holding period after costs, and compares the observed wins with that Poisson-binomial expectation.
- `src/sequence_model.py`: the LSTM baseline in PyTorch, one layer of 48 units reading 20-day sequences built within each ticker. It is trained with Adam on a class-weighted loss, so errors on the rare Buy class cost more, and stops early on the last 15% of the training sequences in date order.
- `src/stationarity.py`: ADF and KPSS tests on each feature, combined into one verdict.
- `src/instrument_validation.py` and `src/llm_evaluation.py`: Cronbach's alpha, band sensitivity, the suitability-matrix checks and the contradiction filter's error rates.

The application itself downloads live prices, so its backtest figures can differ slightly from those in Chapter 5.

CHAPTER 5: EVALUATION (2365/2500 words)

5.1 Data Leakage Discovery

An earlier version reported a weighted F1 of about 0.84 from a random stratified split and selected Random Forest on that basis. Its gap from a walk-forward signal accuracy of 48.8% suggested the split was invalid. Table 5.1 isolates the two leakage mechanisms of Section 2.5 by varying only the split (on the current snapshot the random-split figure is 0.823):

Splitting strategy	Weighted F1	Random baseline	n test
Random stratified (originally reported)	0.823	0.346	4,712
Chronological	0.386	0.342	4,712
Chronological with purged label overlap	0.363	0.345	4,712

Table 5.1: Weighted F1 of the Random Forest under three splitting strategies

The originally reported figure was therefore not an estimate of out-of-sample performance: under correct evaluation the same model scores 0.363 against a random baseline of 0.345, effectively no skill. Abandoning the random split costs 0.438 F1 and purging a further 0.023, so adjacent-row leakage dominates, and the effect persists on the S&P 100 (0.742 to 0.359; Section 5.6). This diagnostic splits rows at the 80th percentile ($n = 4,712$); the production split (Section 4.3) splits by date and holds out 4,720 rows.

5.2 Model Comparison

All models were retrained on the purged split and scored against naive baselines on exactly the rows the LSTM can predict ($n = 4,340$); only the LSTM's loss is class-weighted (Section 4.8). Accuracy carries 95% Wilson intervals.

Model	Accuracy [95% CI]	Weighted F1	Macro F1	Balanced acc.
Logistic Regression	0.401 [0.387, 0.416]	0.371	0.337	0.358
Random Forest	0.384 [0.370, 0.398]	0.365	0.342	0.356
Decision Tree	0.383 [0.369, 0.398]	0.373	0.349	0.356
LSTM (20-day sequences)	0.384 [0.370, 0.399]	0.381	0.365	0.363

Baseline: majority class	0.362 [0.347, 0.376]	0.192	0.177	0.333
Baseline: random	0.350 [0.336, 0.365]	0.350	0.330	0.331

Table 5.2: Classifiers and naive baselines on the same 4,340 test rows

Paired McNemar tests on the same rows (Dietterich, 1998):

Comparison	p-value	Conclusion
LSTM vs Logistic Regression	0.017	Logistic Regression significantly better (this LSTM run)
LSTM vs Random Forest	1.000	No significant difference
LSTM vs random baseline	0.001	LSTM significantly better
Logistic Regression vs Random Forest	0.062	No significant difference
Logistic Regression vs Decision Tree	0.061	No significant difference
Logistic Regression vs majority baseline	<0.001	Logistic Regression significantly better
Logistic Regression vs random baseline	<0.001	Logistic Regression significantly better

Table 5.3: Paired McNemar tests

The ranking depends on which metric is chosen, and that disagreement is the most informative result. Logistic regression has the highest accuracy and McNemar confirms its error rate is significantly lower than the LSTM's ($p = 0.017$), yet the LSTM leads on weighted F1, macro F1 and balanced accuracy. The per-class breakdown explains why: logistic regression almost never predicts the minority Buy class, whose recall is 0.07 against a precision of 0.54. Its accuracy comes from predicting the two frequent classes. A tool meant to surface Buy candidates that finds 7% of them fails at its main task. Retraining the LSTM with five random seeds shows which difference is robust: its macro F1 beats Logistic Regression's in every run (0.365–0.381), but its accuracy ranges from 0.380 to 0.408, so the accuracy ranking depends on the seed.

As Section 2.1 anticipates for a close-to-efficient market, four models spanning linear, tree and sequence architectures converge on only 0.38–0.40 accuracy against 0.35–0.36 for the baselines, consistent with a hard labelling problem rather than one poorly specified model. Nor is the +10% threshold the cause: across Buy thresholds from +5%

to +15% (Appendix H), Logistic Regression's margin over the majority baseline never exceeds the 4.6 points it reaches at +10%, and at +15% it falls below the baseline.

5.3 Stationarity Analysis

The row-wise models assume a stable joint distribution. ADF and KPSS were applied to each feature; their nulls are opposed, so agreement is stronger evidence than either alone.

Feature	ADF p	KPSS p	Verdict
close price	0.549	0.010	Non-stationary
ma 5	0.496	0.010	Non-stationary
ma 20	0.404	0.010	Non-stationary
ma 50	0.391	0.010	Non-stationary
volatility	0.000	0.010	Conflicting
volume	0.000	0.010	Conflicting
daily return	0.000	0.100	Stationary
RSI	0.000	0.100	Stationary
MACD	0.000	0.100	Stationary

Table 5.4: Stationarity tests on each feature

Six of nine features are not clearly stationary. Price levels and moving averages trend, whereas differenced and bounded features are well behaved. A model fitted to 2021 price levels is extrapolating at 2024 levels, which likely contributes to the regime-dependent accuracy in Section 5.5, although this was not tested directly.

5.4 Backtesting Results

Classification accuracy and investment value are distinct questions. The backtester retrains on each rolling window (500-day train, 60-day test), simulates the signals with transaction costs and compares against buy-and-hold across all twenty tickers.

Metric	Result	Interpretation
Pooled win rate	169/272 = 62.1% [56.2%, 67.7%]	Above a coin flip ($p < 0.001$) and the 46.7% expected from matched random entries ($z = 5.34$, $p < 0.001$)
Mean strategy return	+23.90%	Positive but highly dispersed across tickers
Mean benchmark return	+63.72%	Buy-and-hold over the same period
Mean excess return	-39.82% ($p = 0.043$)	Significant underperformance at $n = 20$ tickers
Sharpe ratios	18 of 20 intervals include zero; NVDA and WMT lie above it	Risk-adjusted return mostly indistinguishable from no skill

Tickers with no Buy trades	1 of 20 (JNJ)	Never issued a Buy; its Hold half-positions returned +9.6% against a +4.3% benchmark
----------------------------	---------------	--

Table 5.5: Walk-forward backtest over the 20 stocks (Logistic Regression, corrected costs)

These figures use corrected cost accounting. An earlier backtester charged a Hold half-position's round-trip costs on every day it stayed open, reporting a mean strategy return of +6.46% and excess return of −57.25% ($p = 0.008$). Charging costs only when a position opens or closes (Section 4.5) leaves the win rate unchanged and narrows the shortfall to 39.82 points ($p = 0.043$).

Per-ticker rates show why pooling is necessary: the best was PEP at 100% from five trades (interval 43 points wide), the worst HD at 16.7% from six. Power analysis: the pooled 62.1% needs 126 trades and has 272, whereas detecting a smaller but still valuable 55% edge would need 778. No single ticker comes close, so the pooled figure carries the claim. Appendix B lists every ticker's results.

Three statements are simultaneously true, and only intervals make that visible: the bot picks profitable trades more often than chance; its risk-adjusted return is mostly indistinguishable from zero; and it significantly trails passive holding. The first concerns signal quality and the third investment value, and conflating them is the error this evaluation was restructured to avoid.

5.5 Market Regime Analysis

For each regime a fresh model was trained on all preceding data, keeping every evaluation out-of-sample.

Regime	Train n	Test n	Accuracy [95% CI]
2021 bull market	3,240	5,040	0.437 [0.424, 0.451]
2022 bear market	8,280	5,020	0.318 [0.305, 0.331]
2023-24 recovery	13,320	9,420	0.405 [0.395, 0.415]

Table 5.6: Accuracy by market regime

One-way ANOVA gives $F = 83.02$, $p < 0.001$, and Kruskal-Wallis $p < 0.001$; all three Tukey HSD (Tukey, 1949) pairs survive family-wise correction, the largest gap being

0.119 between 2021 and 2022. Because per-row accuracy is binary, the rank test takes precedence. Each regime's model trains on all earlier data, so the 2021 model has only eight months of 2020 data ($n = 3,240$), less than its own test period.

The substantive finding is that 2022 bear-market accuracy falls to 0.318, with an upper bound of 0.331 sitting below the 0.333 random baseline. The model performs worst precisely when avoiding losses matters most, a material limitation for an advisory system consistent with the non-stationarity in Section 5.3.

5.6 Scaling to 100 Stocks

Twenty stocks is small for a recommender, and three usability participants asked for more (Section 5.8). `experiments/exp07_universe_scaling.py` therefore repeats the core analyses on the S&P 100 (Section 3.5). Two constituents lack a usable 2020–2024 history (Sandisk and GE Vernova), leaving 98 stocks and 115,256 labelled rows, five times the core data. Both universes were run on the same data snapshot (Table 5.7).

Result	Core 20	S&P 100 (98 stocks)
Labelled rows (classifier test rows)	23,560 (4,340)	115,256 (21,266)
Weighted F1, random $\rightarrow$ purged split (Random Forest)	0.823 $\rightarrow$ 0.363	0.742 $\rightarrow$ 0.359
Logistic Regression accuracy [95% CI]	0.401 [0.387, 0.416]	0.370 [0.364, 0.377]
Majority-class baseline accuracy	0.362	0.369
Logistic Regression vs majority baseline (McNemar p)	< 0.001	0.61
Macro F1: Logistic Regression / LSTM / random baseline	0.337 / 0.365 / 0.330	0.233 / 0.353 / 0.336
Logistic Regression Buy recall	0.07	0.04
Pooled win rate (chance-matched baseline)	62.1% (46.7%)	61.8% (46.4%)
Mean excess return vs buy-and-hold	−39.8 points ($p = 0.043$)	−35.9 points ($p < 0.001$)
Sharpe intervals including zero	18 of 20	84 of 98
Accuracy 2021 / 2022 / 2023–24	0.437 / 0.318 / 0.405	0.440 / 0.296 / 0.389

Table 5.7: Core results on the 20-stock universe and on the S&P 100

Three findings hold at scale: leakage inflates performance, the strategy trails buy-and-hold and accuracy collapses in the 2022 bear market. One does not: Logistic Regression's edge disappears. On the S&P 100 it is no more accurate than always predicting the most common class ($p = 0.61$), its macro F1 falls below the random baseline's and it almost stops predicting Buy. The LSTM degrades least (macro F1 0.353) but does not significantly beat the majority baseline either ($p = 0.12$). The deployed model's small

edge therefore appears specific to its twenty stocks, which is why the application's universe has not been enlarged: that first needs a model with skill at scale (Section 5.10). Survivorship bias inflates buy-and-hold returns more than those of the partly invested strategy, so the true shortfall is probably smaller than measured.

5.7 Instrument and Filter Validation

Questionnaire reliability was measured on simulated and real responses (Appendix D). The simulated set ($n = 400$) gave Cronbach's $\alpha = 0.910$ with item-total correlations of $0.750\text{--}0.796$. The nine usability participants (Section 5.8) completed the same instrument, giving $\alpha = 0.625$, 95% CI $[0.017, 0.903]$ (Feldt, 1965), below the 0.70 threshold set in Section 3.8. A one-point shift in score cut-offs reclassified 25% of possible scores.

The suitability matrix covers all 15 combinations and is monotonic in risk tolerance and signal strength: internally consistent, but not professionally validated.

The simulation draws every item from one latent trait, so 0.910 is an upper bound by construction. Two features of the real sample explain the shortfall: `investment_goal` had zero variance (all nine participants chose the same option; removing it raises α only to 0.667), and the sample is risk-homogeneous, with seven of nine classified Growth or Aggressive and none Conservative. Restricted range deflates α regardless of item quality (Nunnally and Bernstein, 1994), so internal consistency is unproven rather than disproven.

Without expert review, three failure modes remain plausible: its category boundaries come from score bands rather than a planner's judgement; it treats all three signals as equally reliable, although Buy predictions are far less trustworthy (Section 5.2); and no qualified adviser has checked its explanations. Showing the model's confidence beside each rating is the cheapest mitigation (Extension 7).

The contradiction filter caught all 5 contradictory summaries and passed all 7 sound ones in a 12-case corpus (Appendix E). With so few cases the recall interval is $[0.566, 1.000]$: the filter handles the failure it was built for, but is not shown to be reliable in general. It

checks single-ticker claims only; other statements are backed by the structured data shown alongside.

5.8 Usability Evaluation

The System Usability Scale (SUS) was administered to nine participants after three core tasks (Appendix G). Scores ranged from 47.5 to 92.5, with a mean of 73.6 (SD = 13.6; 95% CI [63.1, 84.1]). This exceeds the 68-point average usability benchmark (Sauro and Lewis, 2016) and falls within Bangor et al.'s (2009) 'good' range. However, as the confidence interval includes 68, the results support that usability is at least average rather than conclusively above average.

Feedback highlighted three issues: limited chatbot scope, unexplained technical indicators, and a restricted universe of twenty equities. The indicator explanations were the most important concern, as non-specialist users struggled to interpret the displayed metrics.

In response, plain-language explanations were added to each stock metric, alongside a clearer signal guide and an educational-use disclaimer (Figure A6). These changes were made after testing, so their effect on usability remains unverified. The chatbot scope remains a limitation, while stock coverage is discussed in Section 5.6.

5.9 Testing

The codebase carries 353 automated tests: Hypothesis property tests for the deterministic components, unit tests for the statistical machinery and integration tests for the pipeline. Statistical functions are checked against independently derived values, and tests added in this revision check the corrected cost accounting, the backtest page's Sharpe interval and the disclaimer on every page (Appendix C).

5.10 Critical Analysis

Successes:

- A working end-to-end system from questionnaire to explained recommendation
- Leakage-free evaluation with every metric referenced to naive baselines
- Model selection by paired significance tests and explicit deployment criteria
- Signal quality and investment value measured separately, shown to diverge
- Core results re-tested on five times as many stocks
- Quantified error rates for the hallucination filter

Limitations:

1. Predictive skill is weak: 0.401 accuracy against a 0.362 majority baseline, and no edge on the S&P 100 (Section 5.6).
2. Buy recall is 0.07; the model rarely finds the class the tool exists to surface.
3. The strategy trails buy-and-hold by 39.8 points ($p = 0.043$; 35.9 points on the S&P 100, $p < 0.001$); on the core universe the gap is significant only at the chosen +10% threshold (Appendix H).
4. Bear-market accuracy (0.318) falls below the 0.333 chance level.
5. Six of nine features are not clearly stationary.
6. Questionnaire reliability on real respondents is $\alpha = 0.625$, below the 0.70 target, from a small, risk-homogeneous sample; the bands are uncalibrated.
7. The suitability matrix and its explanations lack expert review.
8. The filter corpus is small (12 cases), so its recall interval is wide.
9. Both universes carry survivorship bias (Section 3.5).

Extensions:

1. Replace absolute price features with stationary ratios (e.g. price/ma_50).
2. Raise Buy recall through class weighting or probability-threshold tuning.
3. Add regime detection so the system widens caution in bear conditions.
4. Calibrate the score bands and re-estimate α on a larger, risk-diverse sample.
5. Obtain expert review of the suitability matrix and its explanations.
6. Expand the filter corpus to 100+ labelled summaries, including adversarial phrasings.
7. Show the model's confidence beside each rating so users see its uncertainty.
8. Apply the deflated Sharpe ratio (Bailey and López de Prado, 2014) to correct for backtest selection bias.
9. Rebuild the universe from licensed point-in-time constituents, and enlarge the application's universe once a model shows skill at that scale.

CHAPTER 6: CONCLUSION (834/1000 words)

Table 6.1 checks the eight objectives set out in Section 1.3 against the evidence produced for each.

Objective	Evidence	Status
1. Risk questionnaire, five categories	Sections 3.6, 4.4; alpha = 0.910 simulated, 0.625 real (Section 5.7)	Partially achieved: real-responder reliability below the 0.70 target (5.7)
2. Technical indicator feature engineering	Sections 3.5, 4.2; stationarity tested (Section 5.3)	Achieved: six features not clearly stationary (5.3)
3. Train and compare classifiers, leakage-free	Sections 4.3, 4.8, 5.1–5.2, 5.6	Achieved: predictive skill is weak and absent on the S&P 100 (5.6)
4. Suitability engine mapping profiles to signals	Sections 3.6, 4.4; monotonicity validated (5.7)	Achieved: no expert review (5.10)
5. Web application for non-technical users	Section 4.7; SUS study (Section 5.8)	Achieved: SUS 73.6 from nine participants; the interval includes the 68 benchmark (5.8)
6. Walk-forward backtesting of recommendation quality	Sections 4.5, 5.4, 5.6	Achieved: strategy underperforms buy-and-hold
7. Statistical inference on all reported results	Sections 4.8, 5.1–5.6 (Wilson intervals, McNemar, ANOVA/Kruskal-Wallis, chance-matched test)	Achieved
8. Validate questionnaire, matrix and LLM filter	Sections 4.8, 5.7–5.8	Partially achieved: all three validated; questionnaire reliability below threshold, no expert review, 12-case filter corpus

Table 6.1: Objectives against evidence

This project set out to build and evaluate an AI-powered financial advisor that combines risk profiling with machine learning stock predictions. The system assigns a risk category from five behavioural questions and rates stocks Suitable, Use Caution or Not Suitable,

with charts, indicators and explanations; a chatbot gives access to the same information, with a filter against incorrect LLM statements.

The main finding is that the system's predictive ability is limited and its strategy performs worse than passive investing. Logistic Regression achieves 0.401 accuracy against 0.362 for always predicting the most common class: statistically significant but practically small, and absent when the analysis is repeated on 98 S&P 100 stocks. The backtester records a 62.1% win rate, well above the 46.7% expected from random entries, yet the strategy underperforms buy-and-hold by 39.8 percentage points ($p = 0.043$). Identifying profitable trades more often than chance does not necessarily lead to better overall returns.

One of the most important outcomes of the project was methodological rather than predictive. A random split increased weighted F1 from 0.363 to 0.823, demonstrating how strongly data leakage can inflate reported performance. This supports the concerns raised by Kapoor and Narayanan (2023) and suggests that unusually high results in financial prediction should be treated cautiously when leakage controls are unclear. The revision added a second lesson. A cost-accounting error in the backtester had overstated the strategy's shortfall by about 17 points, and the backtest page showed a Sharpe interval over ten times too narrow: evaluation code needs the same testing as application code.

The project also shows the importance of separating signal quality from investment value. Classification metrics such as accuracy and F1 can indicate predictive performance, but they do not show whether following the predictions would outperform a passive strategy. Including transaction costs, confidence intervals and statistical testing in the backtester made this difference clear.

More broadly, the system works well as an engineering prototype by providing personalised, explainable and auditable recommendations, but it is not yet reliable enough to be treated as investment advice. This highlights the importance of

communicating limitations clearly rather than presenting favourable metrics without their wider context.

Should the system be used for trading at all? On the evidence in Chapter 5, no. That is not the same as having no value. It still works as a risk-profiling and financial-literacy tool: it makes users articulate their risk tolerance, shows how a suitability judgement is derived rather than asserted, and makes visible, through the backtester's intervals, that beating chance on signal quality can still lose to doing nothing. Its honest value is educational, and that is how it should be presented.

Because its output reads as financial advice, the system carries ethical obligations beyond accuracy. FINRA's (2014) suitability principle shaped the rule-based Suitability Engine, whose 15 outcomes can be enumerated and tested (Section 5.7), and every page now carries a disclaimer that the system is an educational prototype, not financial advice (Section 5.8). The larger residual risk sits upstream: a confidently wrong Buy signal is more dangerous than an uncertain one, which is why Section 5.10 treats the low Buy recall as a first-order limitation.

Future work should replace non-stationary price features with more stable alternatives such as price-to-moving-average ratios, and add regime detection so the system becomes more cautious in conditions like the 2022 bear market. A larger, risk-diverse sample is needed to establish the questionnaire's reliability, and expert review would strengthen the suitability matrix. The application's universe should be enlarged only once a model shows skill at that scale, ideally on a point-in-time constituent list that removes survivorship bias. Larger user studies and a broader contradiction-filter test set would further evaluate usability and reliability.

Overall, the project achieved its engineering objective of building a personalised and explainable financial decision-support system, but the evaluation shows that its predictive component still requires improvement before it can provide dependable investment guidance.

CHAPTER 7: APPENDICES

Appendix A: Web Application Interface

Figure A1: Risk tolerance questionnaire

Answer 5 questions and we'll recommend stocks suited to your risk profile

 **Risk Tolerance Assessment**

Answer these scenario-based questions to determine your investor profile.

Q1. If your portfolio dropped 20% in a month, what would you most likely do?

☐ Sell everything to prevent further losses

☐ Sell some holdings to reduce risk

☐ Hold and wait for recovery

☐ Buy more at the lower prices

Q2. What is your primary investment objective?

☐ Preserve my capital — I cannot afford to lose money

☐ Generate steady income with minimal risk

☐ Balanced growth — willing to accept moderate ups and downs

☐ Aggressive growth — maximise returns, accept large swings

Q3. When do you expect to need this invested money?

☐ Within 1 year

☐ 1 to 3 years

☐ 3 to 7 years

☐ More than 7 years

Q4. How would you feel if a single stock in your portfolio lost 30% of its value?

☐ I would lose sleep and want to sell immediately

☐ Very uncomfortable — I'd seriously consider selling

☐ Somewhat uncomfortable but I'd hold on

☐ Fine — losses are part of investing

Q5. Which portfolio outcome would you prefer over one year?

☐ Gain +3% guaranteed, no chance of loss

☐ Likely gain +5%, possible loss of -2%

☐ Likely gain +12%, possible loss of -10%

☐ Likely gain +25%, possible loss of -20%

[Get My Stock Recommendations](#)

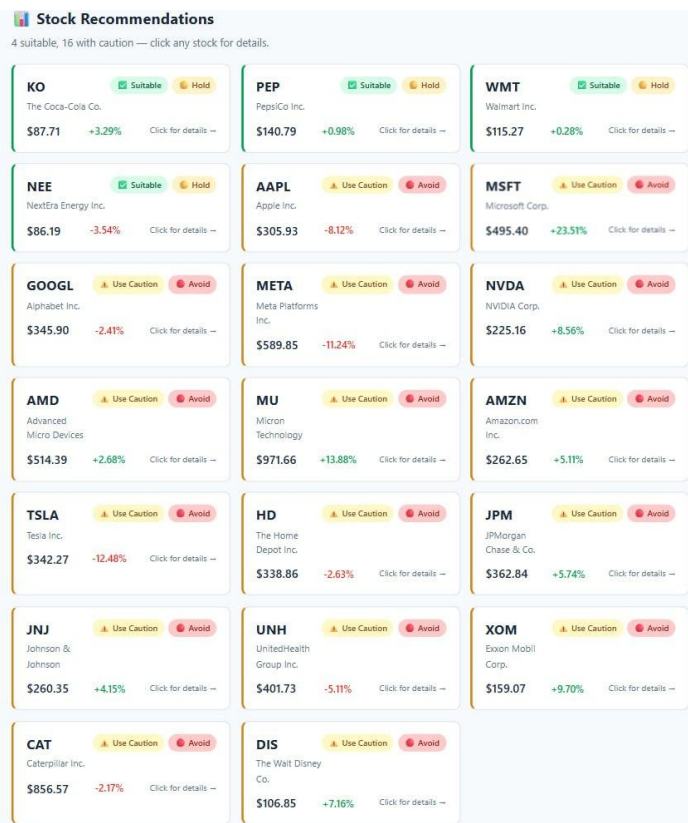
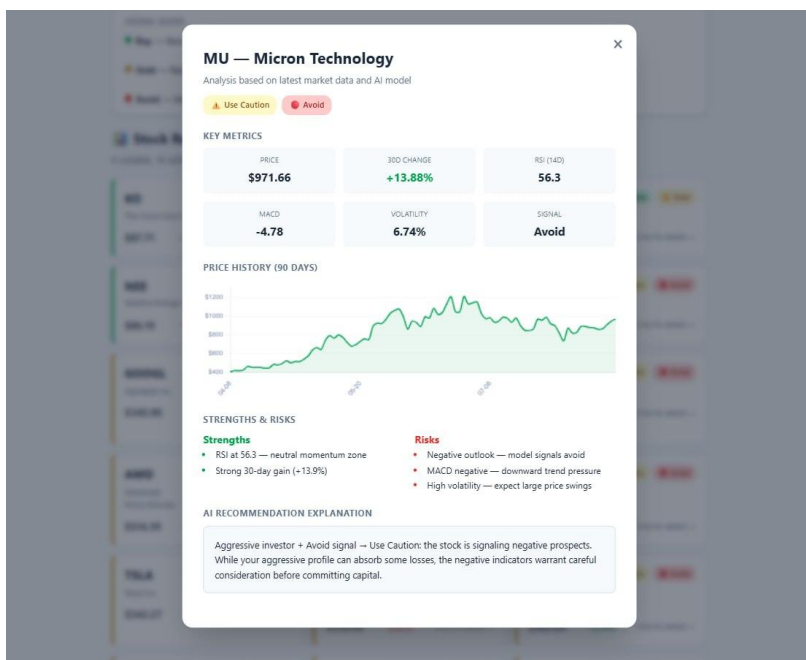
Figure A2: Recommendations with risk profile summary*Figure A3: Stock detail with chart and explanation*

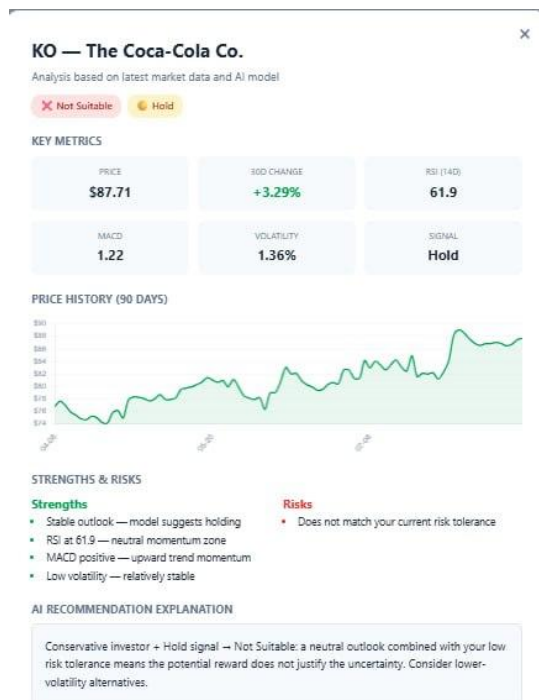
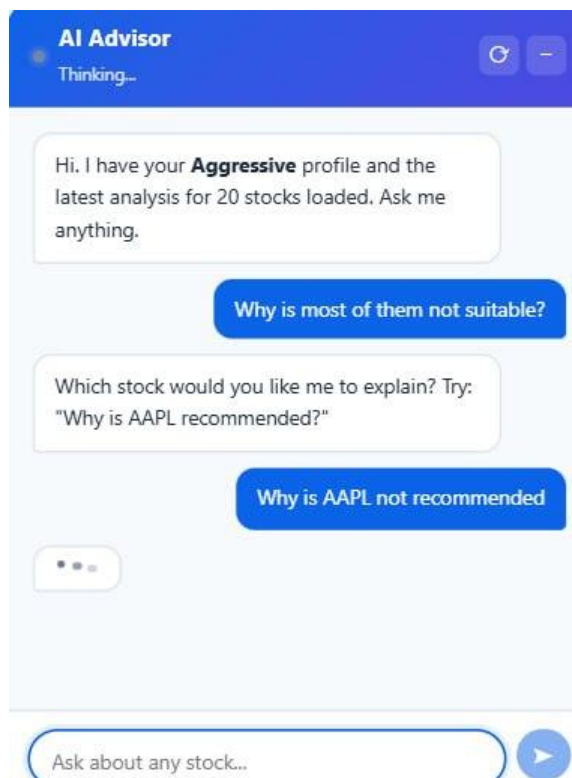
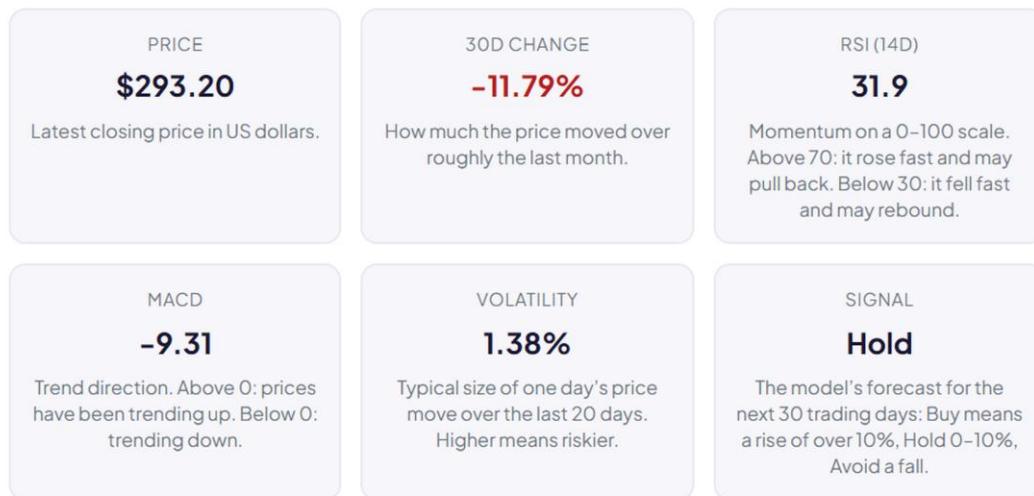
Figure A4: Stock flagged Not Suitable (For a conservative investors)*Figure A5: Conversational assistant with AI summary*

Figure A6: Disclaimer and plain-language indicator explanations added after the usability evaluation

(a) Disclaimer shown at the top of every page

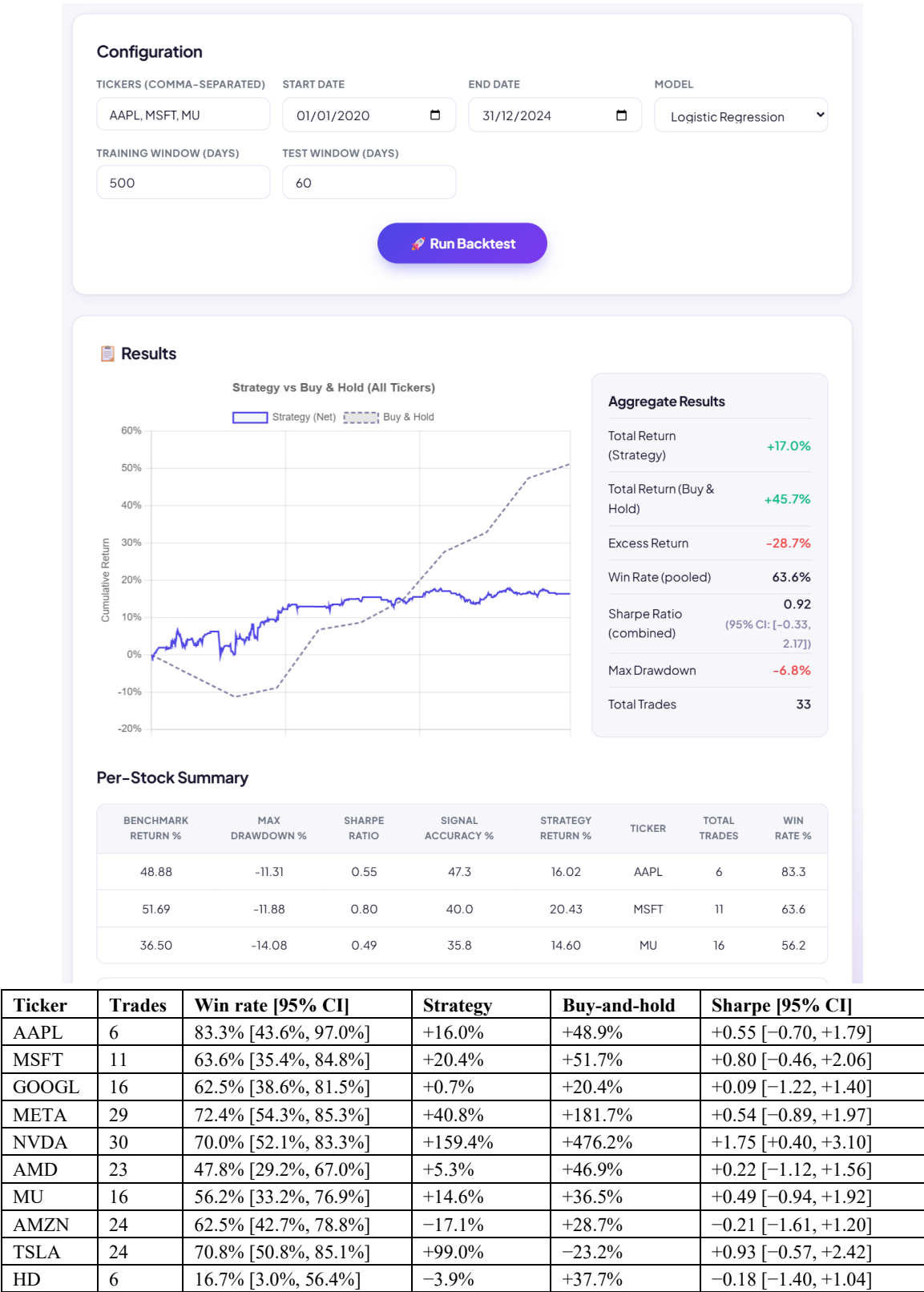


(b) Plain-language explanation under each metric in the stock detail view



Appendix B: Backtesting Evidence

Figure B1: Backtest page for AAPL, MSFT and MU, with the corrected cost accounting and Sharpe interval



Ticker	Trades	Win rate [95% CI]	Strategy	Buy-and-hold	Sharpe [95% CI]
KO	4	75.0% [30.1%, 95.4%]	+4.4%	+29.6%	+0.31 [-0.94, +1.56]
PEP	5	100.0% [56.6%, 100.0%]	+12.8%	+19.5%	+0.78 [-0.46, +2.02]
WMT	3	66.7% [20.8%, 93.9%]	+32.0%	+79.2%	+1.42 [+0.17, +2.66]
JPM	4	25.0% [4.6%, 69.9%]	+11.4%	+81.6%	+0.40 [-0.82, +1.63]
JNJ	0	no Buy trades	+9.6%	+4.3%	+0.54 [-0.67, +1.76]
UNH	6	100.0% [61.0%, 100.0%]	+31.5%	+25.3%	+1.14 [-0.10, +2.38]
XOM	10	60.0% [31.3%, 83.2%]	+35.8%	+46.0%	+0.92 [-0.35, +2.18]
CAT	8	37.5% [13.7%, 69.4%]	+12.3%	+96.0%	+0.65 [-0.61, +1.91]
DIS	35	54.3% [38.2%, 69.5%]	+6.2%	-26.4%	+0.25 [-1.28, +1.78]
NEE	12	58.3% [32.0%, 80.7%]	-13.4%	+13.6%	-0.47 [-1.74, +0.80]

Table B1: Walk-forward results for every ticker (Logistic Regression, corrected costs).
Win-rate intervals are Wilson intervals; Sharpe intervals follow Lo (2002).

Appendix C: Test Evidence

Figure C1: 353 automated tests passing

```
PS C:\Github_projects\FYP_Project> python -m pytest -q
...
warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 353 passed, 5 warnings in 20.82s =====
```

Appendix D: Questionnaire Reliability Analysis

Simulated responses. Cronbach's alpha computed on 400 simulated respondents at each coherence level. Latent coherence is the weight each simulated answer gives to the respondent's underlying risk tolerance rather than to random noise.

Latent coherence	Cronbach's alpha	Interpretation
0.5	0.759	Acceptable
0.7	0.910	Excellent
0.9	0.984	Excellent

Table D1: Cronbach's alpha on 400 simulated respondents by latent coherence

Real respondents. The nine usability participants' answers, analysed in Section 5.7.

Participant	Loss	Goal	Horizon	Comfort	Trade-off	Total	Category
P1	3	3	3	2	2	13	Balanced
P2	4	3	3	3	4	17	Growth
P3	3	3	3	3	3	15	Growth

P4	4	3	3	4	3	17	Growth
P5	4	3	4	3	3	17	Growth
P6	4	3	4	3	4	18	Aggressive
P7	4	3	2	3	3	15	Growth
P8	3	3	2	2	3	13	Balanced
P9	3	3	3	4	4	17	Growth

Table D2: Item scores by participant (1 = most risk-averse, 4 = most risk-tolerant).

Item	Mean	SD	Item-rest r	Alpha if dropped
loss_reaction	3.56	0.53	+0.438	0.545
investment_goal	3.00	0.00	+0.000	0.667
time_horizon	3.00	0.71	+0.358	0.591
emotional_comfort	3.00	0.71	+0.507	0.495
risk_return_preference	3.22	0.67	+0.512	0.493

Table D3: Item statistics on real respondents (n = 9). Computed by experiments/exp06_real_respondent_reliability.py.

Appendix E: LLM Contradiction Filter Error Rates

Measured on 12 labelled cases generated from real ratings and signals, so ground truth follows by construction. Source: experiments/exp05_instrument_and_llm.py.

Metric	Value	Meaning
Recall	1.000 [0.566, 1.000]	All attributable contradictions caught
Precision	1.000	No sound summary discarded
False accept rate	0.000	No contradiction reached the user
False reject rate	0.000	No valid summary needlessly rejected
Evaluation set	12 cases (5 contradictory, 7 sound)	Generated from real ratings and signals

Table E1: Contradiction filter error rates

Appendix F: Reproducible Experiment Scripts

The scripts in experiments/ and retrain_model.py reproduce every quantitative claim in Chapters 4 and 5 apart from the SUS scores; the repository README lists each script with the report section it supports. They read a dated price snapshot (src/market_data.py, 25 September 2026) that is not redistributed, so fresh downloads may shift some figures slightly.

Appendix G: Usability Evaluation

The System Usability Scale (Brooke, 1996) was completed by nine participants after they had worked through three core tasks: (1) completing the risk questionnaire, (2) viewing a stock's details and (3) comparing two stocks. Participation was voluntary and responses were anonymous; participants are identified as P1–P9 only. Odd-numbered items score as (response – 1) and even-numbered items as (5 – response); the sum is multiplied by 2.5 to give a score out of 100.

Participant	P1	P2	P3	P4	P5	P6	P7	P8	P9
SUS score	70.0	82.5	92.5	87.5	65.0	80.0	47.5	67.5	70.0

Table G1: Individual SUS scores. Mean 73.6, SD 13.6, median 70.0, range 47.5–92.5, 95% CI [63.1, 84.1]. The benchmark for average usability is 68 (Sauro and Lewis, 2016).

Theme	Participants	Representative comment
Chatbot scope too narrow	4	“The AI bot isn’t very helpful, which defeats the purpose of being there to help if I can’t use it to help me.”
Technical terms unexplained	3	“Newcomers might not understand metrics meanings. Would require a brief description of certain terms.”
Universe too small	3	“Only has 20 stocks. Would be nice to have more of the stock market.”

Table G2: Free-text comment themes.

Appendix H: Label-Threshold Sensitivity

Logistic Regression on the purged chronological split (full test set, $n = 4,720$) and the walk-forward backtest, repeated with different Buy thresholds; the Avoid boundary stays at 0%. Accuracy is not comparable across rows because each threshold changes the class balance, so the margin over the majority-class baseline and macro F1 against the random baseline are the comparable measures. Source: experiments/exp08_label_sensitivity.py.

Buy threshold	Buy / Hold / Avoid	LR accuracy	Margin over majority (McNemar p)	Macro F1: LR / random	Buy recall	Trades, win rate	Mean excess return (p)
+5%	39% / 22% / 39%	0.376	+2.2 (<0.001)	0.264 / 0.334	0.22	476, 59.7%	–28.8 (0.099)
+7.5%	30% / 31% / 39%	0.359	+0.4 (0.46)	0.280 / 0.329	0.10	344, 61.0%	–35.2 (0.064)

Buy threshold	Buy / Hold / Avoid	LR accuracy	Margin over majority (McNemar p)	Macro F1: LR / random	Buy recall	Trades, win rate	Mean excess return (p)
+10% (chosen)	22% / 39% / 39%	0.401	+4.6 (<0.001)	0.334 / 0.324	0.07	272, 62.1%	-39.8 (0.043)
+15%	12% / 49% / 39%	0.500	-2.8 (<0.001)	0.334 / 0.327	0.01	184, 62.0%	-31.2 (0.098)

Table H1: Sensitivity of the results to the Buy label threshold (20-stock universe)

CHAPTER 8: REFERENCES

Achelis, S.B. (2001) *Technical Analysis from A to Z*. 2nd edn. New York: McGraw-Hill.

Aroussi, R. (2024) yfinance: download market data from Yahoo! Finance API. Available at: <https://github.com/ranaroussi/yfinance> (Accessed: 10 June 2025).

Bailey, D.H. and López de Prado, M. (2014) 'The deflated Sharpe ratio: correcting for selection bias, backtest overfitting and non-normality', *Journal of Portfolio Management*, 40(5), pp. 94-107.

Bangor, A., Kortum, P. and Miller, J. (2009) 'Determining what individual SUS scores mean: adding an adjective rating scale', *Journal of Usability Studies*, 4(3), pp. 114-123.

Barber, B.M. and Odean, T. (2013) 'The behavior of individual investors', in *Handbook of the Economics of Finance*, Vol. 2. Amsterdam: Elsevier, pp. 1533-1570.

Basak, S., Kar, S., Saha, S., Khaidem, L. and Dey, S.R. (2019) 'Predicting the direction of stock market prices using tree-based ensemble learning', *North American Journal of Economics and Finance*, 47, pp. 552-567.

Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5-32.

Brooke, J. (1996) 'SUS: a quick and dirty usability scale', in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) *Usability Evaluation in Industry*. London: Taylor and Francis, pp. 189-194.

Brown, S.J., Goetzmann, W., Ibbotson, R.G. and Ross, S.A. (1992) 'Survivorship bias in performance studies', *Review of Financial Studies*, 5(4), pp. 553-580.

Chen, T. and Guestrin, C. (2016) 'XGBoost: a scalable tree boosting system', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, pp. 785-794.

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014) 'Learning phrase representations using RNN encoder-decoder for statistical machine translation', in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. Doha: ACL, pp. 1724-1734.

- Cronbach, L.J. (1951) 'Coefficient alpha and the internal structure of tests', *Psychometrika*, 16(3), pp. 297-334.
- D'Acunto, F. and Rossi, A.G. (2021) 'Robo-advising', in *The Palgrave Handbook of Technological Finance*. Cham: Palgrave Macmillan, pp. 725-749.
- Demsar, J. (2006) 'Statistical comparisons of classifiers over multiple data sets', *Journal of Machine Learning Research*, 7, pp. 1-30.
- Dickey, D.A. and Fuller, W.A. (1979) 'Distribution of the estimators for autoregressive time series with a unit root', *Journal of the American Statistical Association*, 74(366), pp. 427-431.
- Dietterich, T.G. (1998) 'Approximate statistical tests for comparing supervised classification learning algorithms', *Neural Computation*, 10(7), pp. 1895-1923.
- Fama, E.F. (1970) 'Efficient capital markets: a review of theory and empirical work', *Journal of Finance*, 25(2), pp. 383-417.
- Feldt, L.S. (1965) 'The approximate sampling distribution of Kuder-Richardson reliability coefficient twenty', *Psychometrika*, 30(3), pp. 357-370.
- FINRA (2014) Rule 2111: Suitability. Financial Industry Regulatory Authority. Available at: <https://www.finra.org/rules-guidance/rulebooks/finra-rules/2111> (Accessed: 10 June 2025).
- Fischer, T. and Krauss, C. (2018) 'Deep learning with long short-term memory networks for financial market predictions', *European Journal of Operational Research*, 270(2), pp. 654-669.
- Fisher, R.A. (1925) *Statistical Methods for Research Workers*. Edinburgh: Oliver and Boyd.
- Grable, J.E. and Lytton, R.H. (1999) 'Financial risk tolerance revisited: the development of a risk assessment instrument', *Financial Services Review*, 8(3), pp. 163-181.
- Gu, S., Kelly, B. and Xiu, D. (2020) 'Empirical asset pricing via machine learning', *Review of Financial Studies*, 33(5), pp. 2223-2273.

Harvey, C.R. and Liu, Y. (2015) 'Backtesting', *Journal of Portfolio Management*, 42(1), pp. 13-28.

Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735-1780.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B. and Liu, T. (2025) 'A survey on hallucination in large language models', *ACM Transactions on Information Systems*, 43(2), pp. 1-55.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A. and Fung, P. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), pp. 1-38.

Kannadhasan, M. (2015) 'Retail investors' financial risk tolerance and their risk-taking behaviour', *IIMB Management Review*, 27(3), pp. 175-184.

Kapoor, S. and Narayanan, A. (2023) 'Leakage and the reproducibility crisis in machine-learning-based science', *Patterns*, 4(9), 100804.

Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. (2012) 'Leakage in data mining: formulation, detection, and avoidance', *ACM Transactions on Knowledge Discovery from Data*, 6(4), pp. 1-21.

Kruskal, W.H. and Wallis, W.A. (1952) 'Use of ranks in one-criterion variance analysis', *Journal of the American Statistical Association*, 47(260), pp. 583-621.

Kwiatkowski, D., Phillips, P.C.B., Schmidt, P. and Shin, Y. (1992) 'Testing the null hypothesis of stationarity against the alternative of a unit root', *Journal of Econometrics*, 54(1-3), pp. 159-178.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S. and Kiela, D. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', in *Advances in Neural Information Processing Systems* 33, pp. 9459-9474.

Lim, B., Arik, S.O., Loeff, N. and Pfister, T. (2021) 'Temporal fusion transformers for interpretable multi-horizon time series forecasting', *International Journal of Forecasting*, 37(4), pp. 1748-1764.

Lo, A.W. (2002) 'The statistics of Sharpe ratios', *Financial Analysts Journal*, 58(4), pp. 36-52.

López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken: Wiley.

Malkiel, B.G. (2003) 'The efficient market hypothesis and its critics', *Journal of Economic Perspectives*, 17(1), pp. 59-82.

McNemar, Q. (1947) 'Note on the sampling error of the difference between correlated proportions or percentages', *Psychometrika*, 12(2), pp. 153-157.

Nti, I.K., Adekoya, A.F. and Weyori, B.A. (2020) 'A systematic review of fundamental and technical analysis of stock market predictions', *Artificial Intelligence Review*, 53, pp. 3007-3057.

Nunnally, J.C. and Bernstein, I.H. (1994) *Psychometric Theory*. 3rd edn. New York: McGraw-Hill.

Patel, J., Shah, S., Thakkar, P. and Kotecha, K. (2015) 'Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques', *Expert Systems with Applications*, 42(1), pp. 259-268.

Rudin, C. (2019) 'Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead', *Nature Machine Intelligence*, 1(5), pp. 206-215.

Sauro, J. and Lewis, J.R. (2016) *Quantifying the User Experience: Practical Statistics for User Research*. 2nd edn. Cambridge, MA: Morgan Kaufmann.

Sezer, O.B., Gudelek, M.U. and Ozbayoglu, A.M. (2020) 'Financial time series forecasting with deep learning: a systematic literature review, 2005-2019', *Applied Soft Computing*, 90, 106181.

Siami-Namini, S., Tavakoli, N. and Siami Namin, A. (2019) 'The performance of LSTM and BiLSTM in forecasting time series', in IEEE International Conference on Big Data. Los Angeles: IEEE, pp. 3285-3292.

Tukey, J.W. (1949) 'Comparing individual means in the analysis of variance', *Biometrics*, 5(2), pp. 99-114.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention is all you need', in *Advances in Neural Information Processing Systems* 30, pp. 5998-6008.

Wikipedia (2026) S&P 100. Available at: https://en.wikipedia.org/wiki/S%26P_100 (Accessed: 25 September 2026).

Wilson, E.B. (1927) 'Probable inference, the law of succession, and statistical inference', *Journal of the American Statistical Association*, 22(158), pp. 209-212.

Zeng, A., Chen, M., Zhang, L. and Xu, Q. (2023) 'Are transformers effective for time series forecasting?', in *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), pp. 11121-11128.